Universidad / Facultad (completar)

Carrera de Ingeniería (completar)

Laboratorio 8.2: Navegación Dinámica con Paso de Argumentos

Patrón Master-Detail con Navigation Compose y Material 3 en la aplicación EcoTrip

2026

Desarrollo de Aplicaciones Móviles · RDA-2 · Grupo 4

James Argüello

Wismar Benítez

Carlos Galarza

Mateo Morales

Ángel Loor

Asignatura (completar)

Docente (completar)

Octubre de 2026

Introducción

Casi toda aplicación que muestra datos tiene una pantalla de lista y otra de detalle: el usuario elige un elemento y espera ver, exactamente, la información de ese elemento. Este patrón se conoce como Master-Detail y obliga a resolver una pregunta técnica: ¿cómo sabe la pantalla de destino qué mostrar? En Navigation Compose la respuesta es la ruta: el origen la construye con valores y el destino los recupera del *NavBackStackEntry* (Android Developers, s. f.-a).

Este laboratorio continúa los anteriores. Sobre el flujo Login → Dashboard del Laboratorio 8.1 se añade una lista de asistentes que, al pulsar un elemento, navega a un detalle con el identificador y el nombre del asistente. Se resuelve también el desafío extra, que vuelve opcional el nombre y usa el operador Elvis para un valor por defecto. Las pruebas se hicieron en un emulador con Android 16 (API 36).

Objetivos

1. Configurar rutas de navegación dinámicas con parámetros múltiples.
2. Recuperar y procesar argumentos desde el *NavBackStackEntry*.
3. Mostrar los datos en un componente *Card* de Material 3 con elevación.
4. Verificar la persistencia de los datos del detalle ante la rotación de pantalla.

Marco teórico

Argumentos en la ruta. Una ruta de Navigation Compose puede declarar marcadores entre llaves, como *detail/{asistenteId}*. Los argumentos de ruta (*path*) son obligatorios; los de consulta (*?nombre={asistenteNombre}*) son opcionales y exigen un valor por defecto o aceptar nulos. Cada argumento se declara con *navArgument*, indicando su tipo (*NavType*), si admite nulos y su valor por defecto, y se lee desde *backStackEntry.arguments* (Android Developers, s. f.-a).

Datos mínimos entre pantallas. La guía oficial recomienda pasar solo la información necesaria para que el destino la obtenga, normalmente un identificador, y no objetos completos (Android Developers, s. f.-a). Este principio se retoma en las recomendaciones, pues el laboratorio pasa además el nombre para practicar los parámetros múltiples.

Modelo inmutable. Una *data class* de Kotlin con propiedades *val* genera de forma automática *equals*, *hashCode*, *toString* y *copy*, lo que la hace adecuada para modelar datos que no cambian (JetBrains, s. f.-a).

Nulabilidad y operador Elvis. En Kotlin, un tipo *String?* admite nulos y el operador *?:* devuelve el valor de la derecha si el de la izquierda es nulo, lo que permite declarar un valor por defecto en una sola expresión (JetBrains, s. f.-b).

Card. El componente *Card* de Material 3 agrupa contenido relacionado en una superficie con forma y elevación configurables mediante *CardDefaults.cardElevation* (Android Developers, s. f.-b).

Desarrollo

Adaptaciones respecto a la guía

La guía usa el paquete *com.puce.safepass* y un proyecto con lista de asistentes previa. El repositorio del grupo es EcoTrip 2026 (*com.example.myapplication*) y no contenía esa lista, por lo que se creó junto con el modelo. El modelo se ubicó en *data/model*, junto a los demás modelos del proyecto, en lugar de *models*. El destino se integró al flujo del Laboratorio 8.1: Login → Dashboard → «Ver asistentes» → Detalle.

Paso 1. Modelo de datos

Asistente es una *data class* con *id*, *nombre* y *tipo*, todos *val*. Se agregó una lista de cinco asistentes de ejemplo. Uno se llama «Ana», para reproducir el caso de la guía; otro incluye apóstrofo, y otro contiene «&» y «/», caracteres reservados en una URL, para comprobar que los argumentos viajan intactos.

Paso 2. Pantalla de detalle

DetailScreen recibe *asistenteId* y *asistenteNombre* como *String?* y los muestra en una *Card* con elevación de 8 dp, con las etiquetas «ID de Asistente» y «Nombre». Además de lo pedido, se añadió una barra superior con botón de retorno, para que la pantalla no quede aislada y se pueda volver a la lista desde la interfaz.

Paso 3. Lista clicable

AssistantListScreen muestra los asistentes en un *LazyColumn* con *ListItem*. Cada elemento usa *Modifier.clickable* y llama a la lambda *onAsistenteClick(id, nombre)*, de modo que la pantalla no conoce el *NavController*. Se añadió una clave estable por elemento (*key = id*) para que la lista se recomponga con eficiencia.

Paso 4. Ruta con argumentos en el NavGraph

El archivo *NavGraph.kt* registra dos destinos nuevos. Los fragmentos clave son los siguientes (el listado completo está en el Anexo):

NavGraph.kt: origen de la navegación (lista)

```
        // Destino 4 (Lab 8.2): lista de asistentes (Master).
        composable("list") {
            AssistantListScreen(
                onAsistenteClick = { id, nombre ->
                    // Navegación con argumentos dinámicos. Uri.encode protege los
caracteres
```

```
                    // reservados (espacios, "/", "&", "?") para que no rompan la ruta.
                    navController.navigate("detail/${Uri.encode(id)}?
nombre=${Uri.encode(nombre)}")
                },
                onSinNombreClick = { id ->
                    // Desafío extra: el argumento opcional «nombre» se omite.
                    navController.navigate("detail/${Uri.encode(id)}")
                },
                onVolver = { navController.popBackStack() }
            )
        }
```

NavGraph.kt: destino con argumentos (detalle)

```
        // Destino 5 (Lab 8.2): detalle (Detail). «asistenteId» es obligatorio y
        // «nombre» es un argumento de consulta opcional.
        composable(
            route = "detail/{asistenteId}?nombre={asistenteNombre}",
            arguments = listOf(
                navArgument("asistenteId") { type = NavType.StringType },
                navArgument("asistenteNombre") {
                    type = NavType.StringType
                    nullable = true
                    defaultValue = null
                }
            )
        ) { backStackEntry ->
            // Recuperamos los argumentos del Bundle de navegación.
            val id = backStackEntry.arguments?.getString("asistenteId")
            val nombre = backStackEntry.arguments?.getString("asistenteNombre")

            DetailScreen(
                asistenteId = id,
                asistenteNombre = nombre,
                onVolver = { navController.popBackStack() }
            )
        }
```

La guía propone primero la ruta *detail/{id}/{nombre}* y, en el desafío extra, la versión con nombre opcional *detail/{asistenteId}?nombre={asistenteNombre}*. La entrega usa esta segunda forma, que incluye la primera: con nombre, el comportamiento es idéntico, y sin nombre la ruta sigue siendo válida. El argumento *asistenteId* es obligatorio y *asistenteNombre* es *String* anulable con valor por defecto *null*.

Al construir la ruta se aplica *Uri.encode* a los valores. Sin ese paso, un nombre como «Sofía & Pablo / Equipo QA» introduciría caracteres con significado propio en una URL (espacio, «/», «&»), que pueden alterar cómo se interpreta la ruta. Con la codificación, la prueba recupera el texto completo sin alteraciones (ver Resultados).

Desafío extra. Nombre opcional y operador Elvis

En *DetailScreen*, el texto del nombre se arma con *asistenteNombre ?: "Usuario Invitado"*. Para poder probarlo desde la interfaz, la lista incluye el botón «Abrir detalle sin nombre», que navega a *detail/INVITADO-000* sin el argumento de consulta. Es un artefacto de prueba que se retiraría en una versión final.

Resultados

Se compiló sin errores y se ejecutó la aplicación en el emulador Pixel 8 con Android 16 (API 36). La Figura 1 muestra la lista y la tarjeta de detalle para el asistente «Ana», que es la evidencia exigida por el laboratorio.

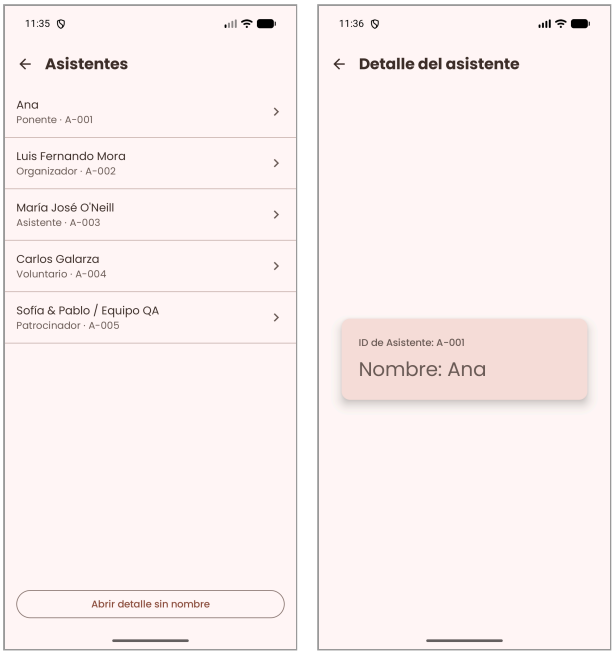


Figura 1. Master-Detail: (izquierda) lista de asistentes; (derecha) tarjeta de detalle tras pulsar «Ana», con «ID de Asistente: A-001» y «Nombre: Ana».

El resultado coincide con el esperado: al pulsar «Ana», la tarjeta muestra «Nombre: Ana». El identificador también llega, lo que confirma que los dos parámetros de la ruta se recuperan correctamente.

Persistencia ante rotación



Figura 2. El detalle de «Ana» tras girar el emulador a horizontal: los datos permanecen visibles.

Al girar el emulador, la *Activity* se recrea, pero los argumentos pertenecen al *NavBackStackEntry*, que el *NavController* restaura, por lo que la tarjeta conserva «Nombre: Ana» (Figura 2). No fue necesario guardar estado adicional: los argumentos de navegación ya son estado persistente de la pantalla.

Casos límite y desafío extra

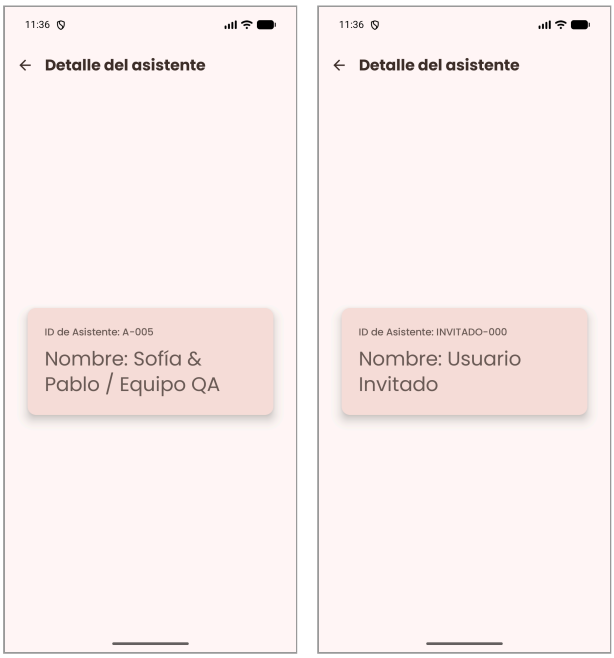


Figura 3. (Izquierda) nombre con «&» y «/» recuperado intacto; (derecha) ruta sin nombre: se muestra «Usuario Invitado» por el operador Elvis.

La Figura 3 resume los dos casos adicionales. A la izquierda, el nombre «Sofía & Pablo / Equipo QA» llega completo gracias a la codificación. A la derecha, la ruta sin el argumento opcional muestra «Nombre: Usuario Invitado» y el identificador «INVITADO-000», verificando el valor por defecto.

Pruebas automatizadas

Se añadió la clase *ArgumentosNavegacionTest* con cuatro pruebas instrumentadas de Compose (Android Developers, s. f.-c), que cubren los resultados esperados de la guía y los casos límite de este informe. La ejecución de *connectedDebugAndroidTest* en el emulador API 36 terminó con 12 de 12 pruebas aprobadas.

Tabla 1

Pruebas instrumentadas del Laboratorio 8.2 y su resultado en el emulador (API 36)

Prueba	Qué verifica	Resultado
clicEnAna_muestraTarjetaConNombreAna	«Nombre: Ana» e ID «A-001» en la tarjeta	Aprobada
nombreConCaracteresReservados_seRecuperaIntacto	Nombre con «&» y «/» sin alteraciones	Aprobada
rotacion_conservaDatosDelDetalle	Los datos permanecen tras recrear la Activity	Aprobada
sinNombre_muestraUsuarioInvitado	Valor por defecto con el operador Elvis	Aprobada

El total de 12 pruebas incluye las ocho de laboratorios anteriores (resiliencia, flujo de Login y prueba de ejemplo), por lo que la incorporación de la lista no rompió el comportamiento previo.

Análisis

Por qué la ruta es el contrato. La ruta *detail/{asistenteId}?nombre={asistenteNombre}* describe qué necesita el destino y qué es opcional. Esto hace el flujo predecible y depurable, como plantea el contexto del laboratorio: basta con leer la ruta para saber qué recibe la pantalla, sin rastrear variables compartidas.

Argumentos de ruta frente a estado compartido. Una alternativa sería guardar el asistente seleccionado en un *ViewModel* y leerlo desde el detalle. Funciona, pero acopla ambas pantallas a un estado oculto y complica los enlaces profundos y las pruebas. El paso por argumentos mantiene cada pantalla autocontenida.

Duplicación del dato. Pasar el nombre además del identificador replica información que el destino podría obtener del repositorio por su ID. Es didáctico para practicar parámetros múltiples, pero si el nombre cambiara mientras el detalle está abierto, la tarjeta mostraría un valor desactualizado.

Alcance de lo verificado. Los datos son una lista fija de ejemplo; no hay repositorio, red ni carga asíncrona. Los resultados demuestran el paso y la recuperación de argumentos, no el manejo de estados de carga o error de un detalle real.

Conclusiones

El laboratorio mostró que el paso de argumentos en Navigation Compose es sencillo de implementar y, a la vez, tiene detalles que se pasan por alto: los argumentos opcionales requieren valor por defecto o nulabilidad, y los valores con caracteres reservados exigen codificación. Con esas dos precauciones, la ruta *detail/{asistenteId}?nombre={...}* permitió mostrar en una *Card* los datos del asistente elegido, conservarlos al rotar la pantalla y degradar con elegancia cuando falta el nombre.

En mi opinión, lo más valioso fue el caso del nombre con «&» y «/»: la guía no lo menciona y es justo el tipo de dato real que rompe una ruta en producción. Probarlo desde el principio evitó un defecto que, con nombres de prueba sencillos como «Ana», habría pasado inadvertido. También me pareció acertado que la rotación no exigiera código extra, porque confirma que conviene apoyarse en el estado que el framework ya conserva antes de inventar mecanismos propios.

Por último, el laboratorio sugiere una regla práctica: pasar identificadores en la ruta y obtener los datos desde una única fuente. Pasar el nombre es válido como ejercicio, pero en una aplicación con datos cambiantes es una duplicación que conviene evitar.

Recomendaciones

1. Pasar solo el identificador en la ruta y recuperar el resto desde un repositorio, para evitar datos duplicados y desactualizados.

2. Codificar siempre con *Uri.encode* los valores de texto libre incluidos en la ruta, y probar con nombres que contengan espacios, «/», «&», «?» y acentos.

3. Migrar a rutas tipadas serializables (como en el flujo de EcoTrip), que convierten los argumentos en propiedades tipadas y eliminan las cadenas de nombre de argumento.

4. Retirar el botón «Abrir detalle sin nombre» antes de publicar, o moverlo a una variante de depuración.

5. Cuando el detalle dependa de datos remotos, modelar los estados de carga y error con una clase sellada en el *ViewModel*.

6. Mantener pruebas de navegación con argumentos en la integración continua para detectar regresiones al cambiar la ruta.

Referencias

Android Developers. (s. f.-a). *Navigation with Compose*. Google. https://developer.android.com/develop/ui/compose/navigation

Android Developers. (s. f.-b). *Card*. Google. https://developer.android.com/develop/ui/compose/components/card

Android Developers. (s. f.-c). *Testing your Compose layout*. Google. https://developer.android.com/develop/ui/compose/testing

JetBrains. (s. f.-a). *Data classes*. Kotlin Documentation. https://kotlinlang.org/docs/data-classes.html

JetBrains. (s. f.-b). *Null safety*. Kotlin Documentation. https://kotlinlang.org/docs/null-safety.html

Anexo. Código fuente del laboratorio

app/src/main/java/com/example/myapplication/ui/navigation/NavGraph.kt

```
package com.example.myapplication.ui.navigation

import android.net.Uri
import androidx.compose.runtime.Composable
import androidx.navigation.NavType
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import androidx.navigation.compose.rememberNavController
import androidx.navigation.navArgument
import com.example.myapplication.navigation.EcoTripNavHost
import com.example.myapplication.ui.screens.AssistantListScreen
import com.example.myapplication.ui.screens.DashboardScreen
import com.example.myapplication.ui.screens.DetailScreen
import com.example.myapplication.ui.screens.LoginScreen
import com.example.myapplication.ui.viewmodel.EcoTripViewModel

@Composable
fun AppNavigation(ecoTripViewModel: EcoTripViewModel) {
    // 1. El NavController es el "conductor" de la aplicación.
    val navController = rememberNavController()

    // 2. El NavHost es el mapa de rutas; startDestination define la primera pantalla.
    NavHost(navController = navController, startDestination = "auth") {

        // Destino 1: Login
        composable("auth") {
            LoginScreen(
                onLoginSuccess = {
                    // POR QUÉ: se navega a 'main' y se elimina 'auth' de la pila, para que el
                    // botón "Atrás" no devuelva al login.
                    navController.navigate("main") {
                        popUpTo("auth") { inclusive = true }
                    }
                }
            )
        }

        // Destino 2: Dashboard
        composable("main") {
            DashboardScreen(
                onLogout = {
                    navController.navigate("auth") {
                        popUpTo("main") { inclusive = true } // Destruye el dashboard
                    }
                },
                onAbrirEcoTrip = { navController.navigate("ecotrip") },
                onAbrirAsistentes = { navController.navigate("list") }
            )
        }

        // Destino 3: flujo EcoTrip (formulario y resumen). "Atrás" regresa al Dashboard.
        composable("ecotrip") {
            EcoTripNavHost(viewModel = ecoTripViewModel)
        }

        // Destino 4 (Lab 8.2): lista de asistentes (Master).
        composable("list") {
            AssistantListScreen(
                onAsistenteClick = { id, nombre ->
                    // Navegación con argumentos dinámicos. Uri.encode protege los caracteres
                    // reservados (espacios, "/", "&", "?") para que no rompan la ruta.
                    navController.navigate("detail/${Uri.encode(id)}?nombre=${Uri.encode(nombre)}")
                },
                onSinNombreClick = { id ->
                    // Desafío extra: el argumento opcional «nombre» se omite.
                    navController.navigate("detail/${Uri.encode(id)}")
                },
                onVolver = { navController.popBackStack() }
```

```
            )
        }

        // Destino 5 (Lab 8.2): detalle (Detail). «asistenteId» es obligatorio y
        // «nombre» es un argumento de consulta opcional.
        composable(
            route = "detail/{asistenteId}?nombre={asistenteNombre}",
            arguments = listOf(
                navArgument("asistenteId") { type = NavType.StringType },
                navArgument("asistenteNombre") {
                    type = NavType.StringType
                    nullable = true
                    defaultValue = null
                }
            )
        ) { backStackEntry ->
            // Recuperamos los argumentos del Bundle de navegación.
            val id = backStackEntry.arguments?.getString("asistenteId")
            val nombre = backStackEntry.arguments?.getString("asistenteNombre")

            DetailScreen(
                asistenteId = id,
                asistenteNombre = nombre,
                onVolver = { navController.popBackStack() }
            )
        }
    }
}
```

app/src/main/java/com/example/myapplication/data/model/Asistente.kt

```
package com.example.myapplication.data.model

// POR QUÉ: data class para aprovechar la inmutabilidad y el método copy().
data class Asistente(
    val id: String,
    val nombre: String,
    val tipo: String
)

/** Datos de ejemplo para la lista Master-Detail (Lab 8.2). */
val asistentesDemo = listOf(
    Asistente(id = "A-001", nombre = "Ana", tipo = "Ponente"),
    Asistente(id = "A-002", nombre = "Luis Fernando Mora", tipo = "Organizador"),
    Asistente(id = "A-003", nombre = "María José O'Neill", tipo = "Asistente"),
    Asistente(id = "A-004", nombre = "Carlos Galarza", tipo = "Voluntario"),
    Asistente(id = "A-005", nombre = "Sofía & Pablo / Equipo QA", tipo = "Patrocinador")
)
```

app/src/main/java/com/example/myapplication/ui/screens/AssistantListScreen.kt

```
package com.example.myapplication.ui.screens

import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.automirrored.filled.ArrowBack
import androidx.compose.material.icons.automirrored.filled.KeyboardArrowRight
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.HorizontalDivider
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.ListItem
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedButton
import androidx.compose.material3.Scaffold
```

```
import androidx.compose.material3.Text
import androidx.compose.material3.TopAppBar
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import com.example.myapplication.data.model.Asistente
import com.example.myapplication.data.model.asistentesDemo

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun AssistantListScreen(
    onAsistenteClick: (id: String, nombre: String) -> Unit,
    onSinNombreClick: (id: String) -> Unit,
    onVolver: () -> Unit,
    asistentes: List<Asistente> = asistentesDemo
) {
    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("Asistentes") },
                navigationIcon = {
                    IconButton(onClick = onVolver) {
                        Icon(
                            imageVector = Icons.AutoMirrored.Filled.ArrowBack,
                            contentDescription = "Volver al Dashboard"
                        )
                    }
                }
            )
        }
    ) { innerPadding ->
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(innerPadding)
        ) {
            LazyColumn(modifier = Modifier.weight(1f)) {
                items(asistentes, key = { it.id }) { asistente ->
                    ListItem(
                        headlineContent = { Text(asistente.nombre) },
                        supportingContent = { Text("${asistente.tipo} · ${asistente.id}") },
                        trailingContent = {
                            Icon(
                                imageVector = Icons.AutoMirrored.Filled.KeyboardArrowRight,
                                contentDescription = null
                            )
                        },
                        modifier = Modifier.clickable {
                            // Esta lambda se define en el NavGraph.
                            onAsistenteClick(asistente.id, asistente.nombre)
                        }
                    )
                    HorizontalDivider()
                }
            }
            // Prueba del desafío extra: ruta de detalle sin el argumento opcional «nombre».
            OutlinedButton(
                onClick = { onSinNombreClick("INVITADO-000") },
                modifier = Modifier
                    .fillMaxWidth()
                    .padding(16.dp)
            ) {
                Text(
                    text = "Abrir detalle sin nombre",
                    style = MaterialTheme.typography.labelLarge
                )
            }
        }
    }
}
```

app/src/main/java/com/example/myapplication/ui/screens/DetailScreen.kt

```
package com.example.myapplication.ui.screens

import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.automirrored.filled.ArrowBack
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.material3.TopAppBar
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun DetailScreen(
    asistenteId: String?,
    asistenteNombre: String?,
    onVolver: (() -> Unit)? = null
) {
    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("Detalle del asistente") },
                navigationIcon = {
                    if (onVolver != null) {
                        IconButton(onClick = onVolver) {
                            Icon(
                                imageVector = Icons.AutoMirrored.Filled.ArrowBack,
                                contentDescription = "Volver a la lista"
                            )
                        }
                    }
                }
            )
        }
    ) { innerPadding ->
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(innerPadding)
                .padding(16.dp),
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.Center
        ) {
            // POR QUÉ: elevación y contenedores son estándar en Material 3.
            Card(
                modifier = Modifier
                    .fillMaxWidth()
                    .padding(16.dp),
                elevation = CardDefaults.cardElevation(defaultElevation = 8.dp)
            ) {
                Column(modifier = Modifier.padding(24.dp)) {
                    Text(
                        text = "ID de Asistente: ${asistenteId ?: "N/A"}",
                        style = MaterialTheme.typography.labelLarge
                    )
                    Spacer(modifier = Modifier.height(8.dp))
                    // Operador Elvis: si el argumento opcional no llega, se usa el valor por defecto.
                    Text(
                        text = "Nombre: ${asistenteNombre ?: "Usuario Invitado"}",
```

```
                        style = MaterialTheme.typography.headlineMedium
                    )
                }
            }
        }
    }
}
```

app/src/main/java/com/example/myapplication/ui/screens/DashboardScreen.kt

```
package com.example.myapplication.ui.screens

import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Button
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedButton
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.unit.dp

@Composable
fun DashboardScreen(
    onLogout: () -> Unit,
    onAbrirEcoTrip: () -> Unit,
    onAbrirAsistentes: () -> Unit
) {
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(24.dp),
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(
            text = "¡Bienvenido al Dashboard Principal!",
            style = MaterialTheme.typography.titleLarge,
            textAlign = TextAlign.Center
        )
        Spacer(modifier = Modifier.height(24.dp))
        Button(onClick = { onAbrirEcoTrip() }) {
            Text("Planificar viaje (EcoTrip)")
        }
        Spacer(modifier = Modifier.height(8.dp))
        Button(onClick = { onAbrirAsistentes() }) {
            Text("Ver asistentes")
        }
        Spacer(modifier = Modifier.height(8.dp))
        OutlinedButton(onClick = { onLogout() }) {
            Text("Cerrar Sesión")
        }
    }
}
```

app/src/androidTest/java/com/example/myapplication/ArgumentosNavegacionTest.kt

```
package com.example.myapplication

import androidx.compose.ui.test.junit4.createAndroidComposeRule
import androidx.compose.ui.test.onNodeWithText
import androidx.compose.ui.test.performClick
import androidx.test.ext.junit.runners.AndroidJUnit4
import org.junit.Before
import org.junit.Rule
import org.junit.Test
```

```
import org.junit.runner.RunWith

/** Lab 8.2: argumentos dinámicos de navegación (Master-Detail). */
@RunWith(AndroidJUnit4::class)
class ArgumentosNavegacionTest {

    @get:Rule
    val composeRule = createAndroidComposeRule<MainActivity>()

    @Before
    fun abrirListaDeAsistentes() {
        composeRule.onNodeWithText("Ingresar al Dashboard").performClick()
        composeRule.onNodeWithText("Ver asistentes").performClick()
    }

    @Test
    fun clicEnAna_muestraTarjetaConNombreAna() {
        composeRule.onNodeWithText("Ana").performClick()
        composeRule.onNodeWithText("Nombre: Ana").assertExists()
        composeRule.onNodeWithText("ID de Asistente: A-001").assertExists()
    }

    @Test
    fun nombreConCaracteresReservados_seRecuperaIntacto() {
        composeRule.onNodeWithText("Sofía & Pablo / Equipo QA").performClick()
        composeRule.onNodeWithText("Nombre: Sofía & Pablo / Equipo QA").assertExists()
    }

    @Test
    fun rotacion_conservaDatosDelDetalle() {
        composeRule.onNodeWithText("Luis Fernando Mora").performClick()
        composeRule.activityRule.scenario.recreate()
        composeRule.onNodeWithText("Nombre: Luis Fernando Mora").assertExists()
        composeRule.onNodeWithText("ID de Asistente: A-002").assertExists()
    }

    @Test
    fun sinNombre_muestraUsuarioInvitado() {
        composeRule.onNodeWithText("Abrir detalle sin nombre").performClick()
        composeRule.onNodeWithText("Nombre: Usuario Invitado").assertExists()
        composeRule.onNodeWithText("ID de Asistente: INVITADO-000").assertExists()
    }
}
```